

## 02 Matrices

# 2.2 Transpose

*The  $A^T$  Notation, Swapping Rows and Columns, and Its Key Properties*

Linear Algebra Made Easy and Visual | Open-source code & drafts: <https://github.com/Visualize-ML>



## Core Skills You Will Master

1

Understand the definition of matrix transpose: swapping rows and columns to get a new matrix, written  $A^T$

3

Understand how transpose changes a matrix's shape: tall  $\leftrightarrow$  wide, square stays square, symmetric stays unchanged

5

Understand transpose from a geometric view: how row and column vectors, as arrows, swap the spaces they live in

7

Master the key properties of transpose: double transpose restores the original matrix, it commutes with addition, and is unaffected by scalar multiplication

2

Master vector transpose: a row vector transposes into a column vector, and a column vector into a row vector

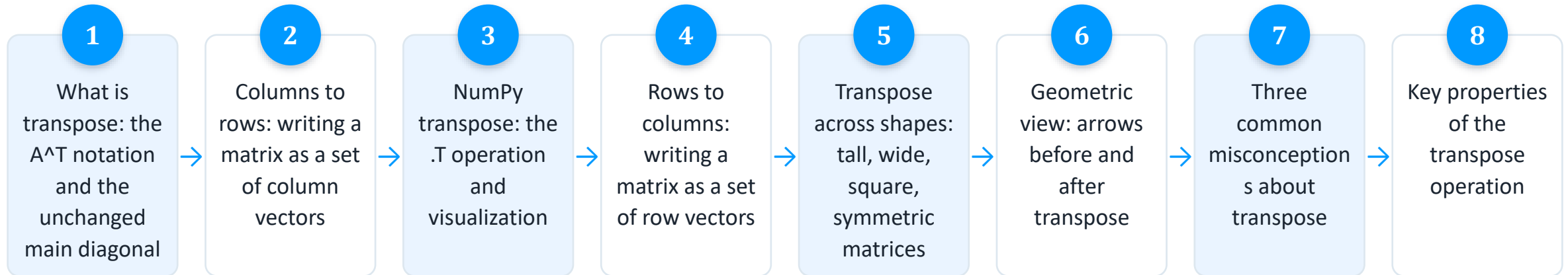
4

Use NumPy's `.T` fluently to get the transpose of a 2D array, and visualize the change with heatmaps

6

Avoid three common misconceptions about transpose: it is not a diagonal mirror, not a left-right mirror, not an up-down mirror

# How This Section Unfolds



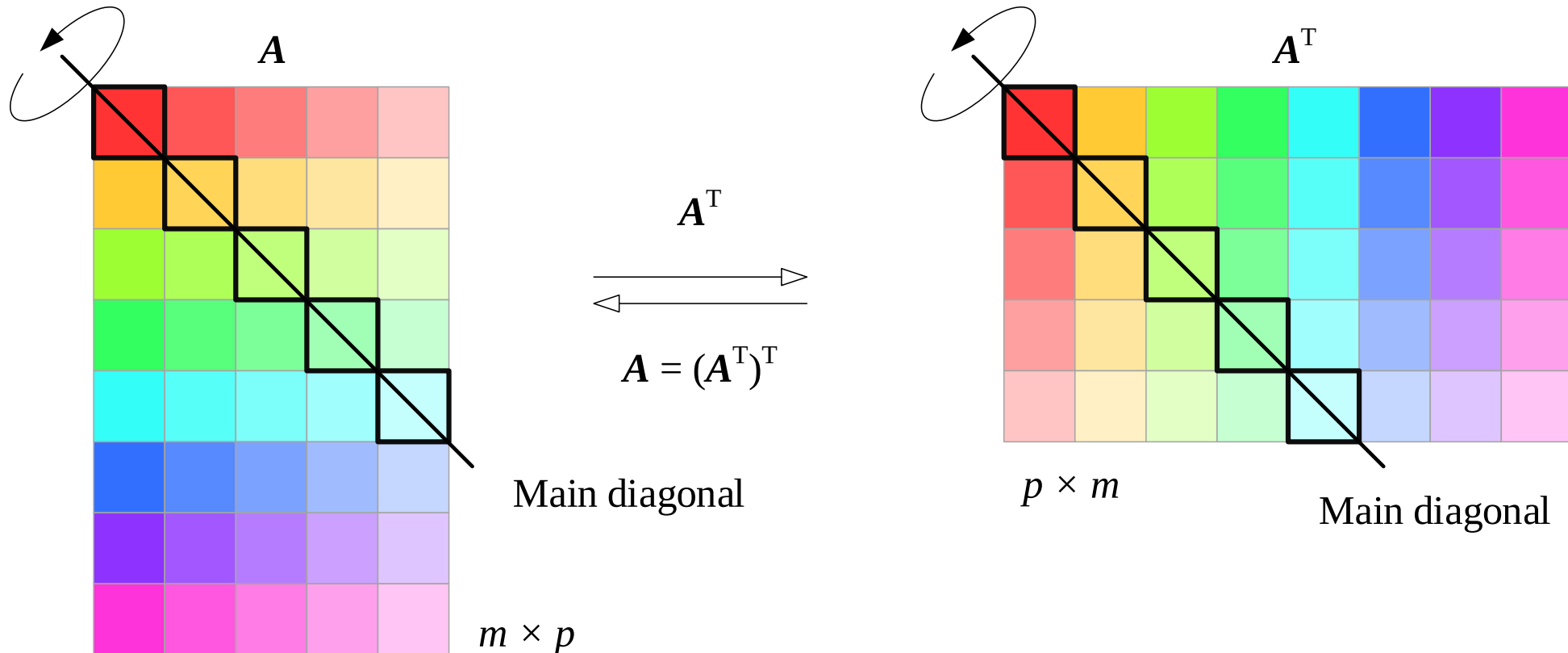
## What Is a Transpose?

*The transpose of matrix  $\mathbf{A}$  is written  $\mathbf{A}^T$ : swap its rows and columns to get a new matrix*

- A column vector transposes into a row vector, and vice versa — transpose is a "matrix  $\rightarrow$  matrix" operation
- NumPy uses `.T` for array transpose; this book consistently writes it as  $\mathbf{A}^T$  (never  $\mathbf{A}'$ )
- An  $m \times p$  matrix  $\mathbf{A}$  transposes into a  $p \times m$  matrix  $\mathbf{A}^T$ ; the diagonal entries  $a_{i,i}$  stay in place

## Figure 1 · Transpose Leaves the Main Diagonal Unchanged

- Matrix  $A$  mirrors around its main diagonal to become  $A^T$ ; the diagonal entries never move
- An  $m \times p$  matrix  $A$  transposes into a  $p \times m$  matrix  $A^T$



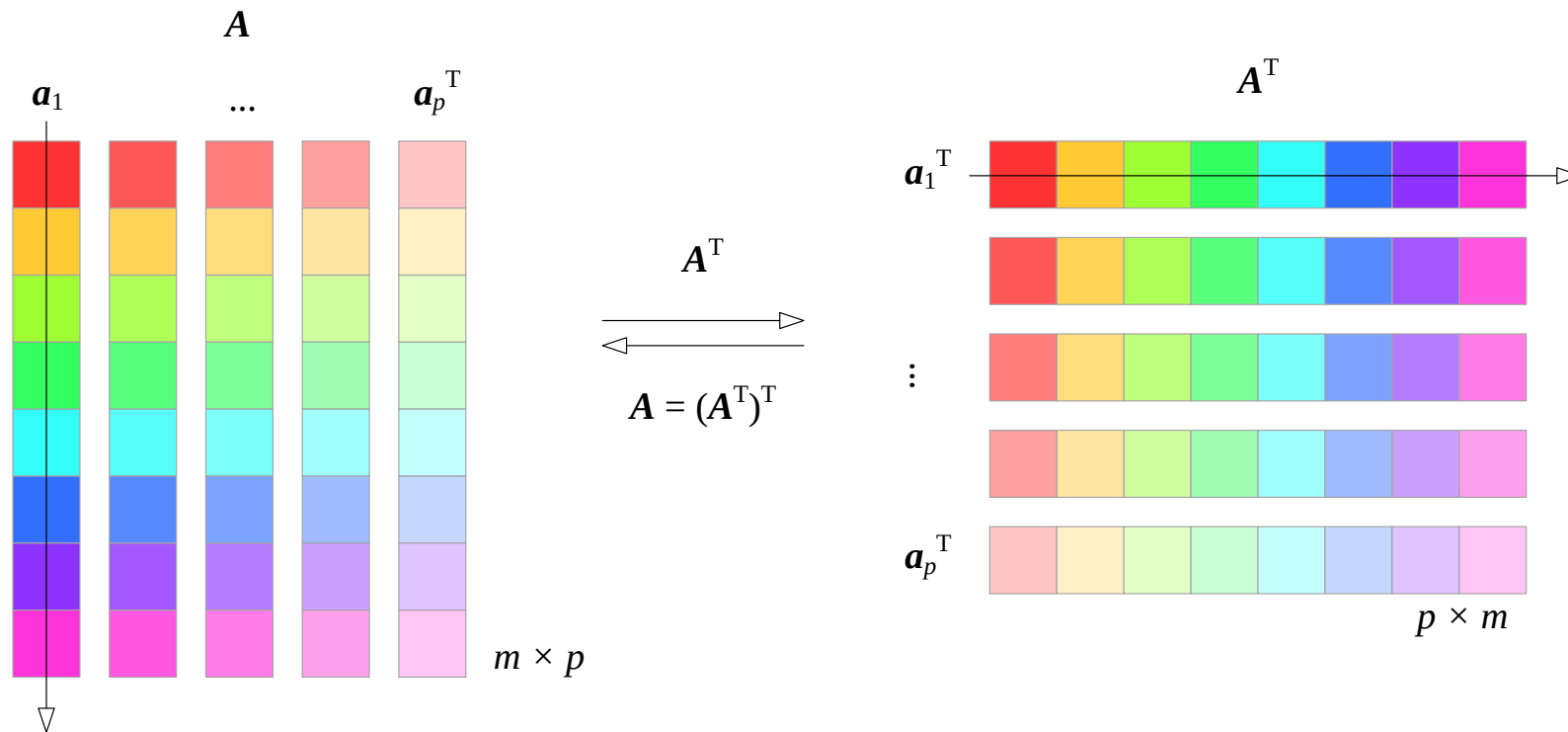
## Writing a Matrix as a Set of Column Vectors

$$A = [\mathbf{a}_1 \ \mathbf{a}_2 \ \cdots \ \mathbf{a}_p], \text{ so } A^T = [\mathbf{a}_1^T; \mathbf{a}_2^T; \cdots; \mathbf{a}_p^T]$$

- Once A is written as a set of column vectors, transpose turns each column vector into the matching row vector
- Column 1 of A becomes row 1 of  $A^T$ , column 2 becomes row 2, and so on — the relative order of entries never changes

## Figure 2 · Transpose Turns Columns into Rows

- Every original column vector becomes the matching row vector in  $A^T$
- The relative order of entries — the shading gradient in the diagram — stays the same



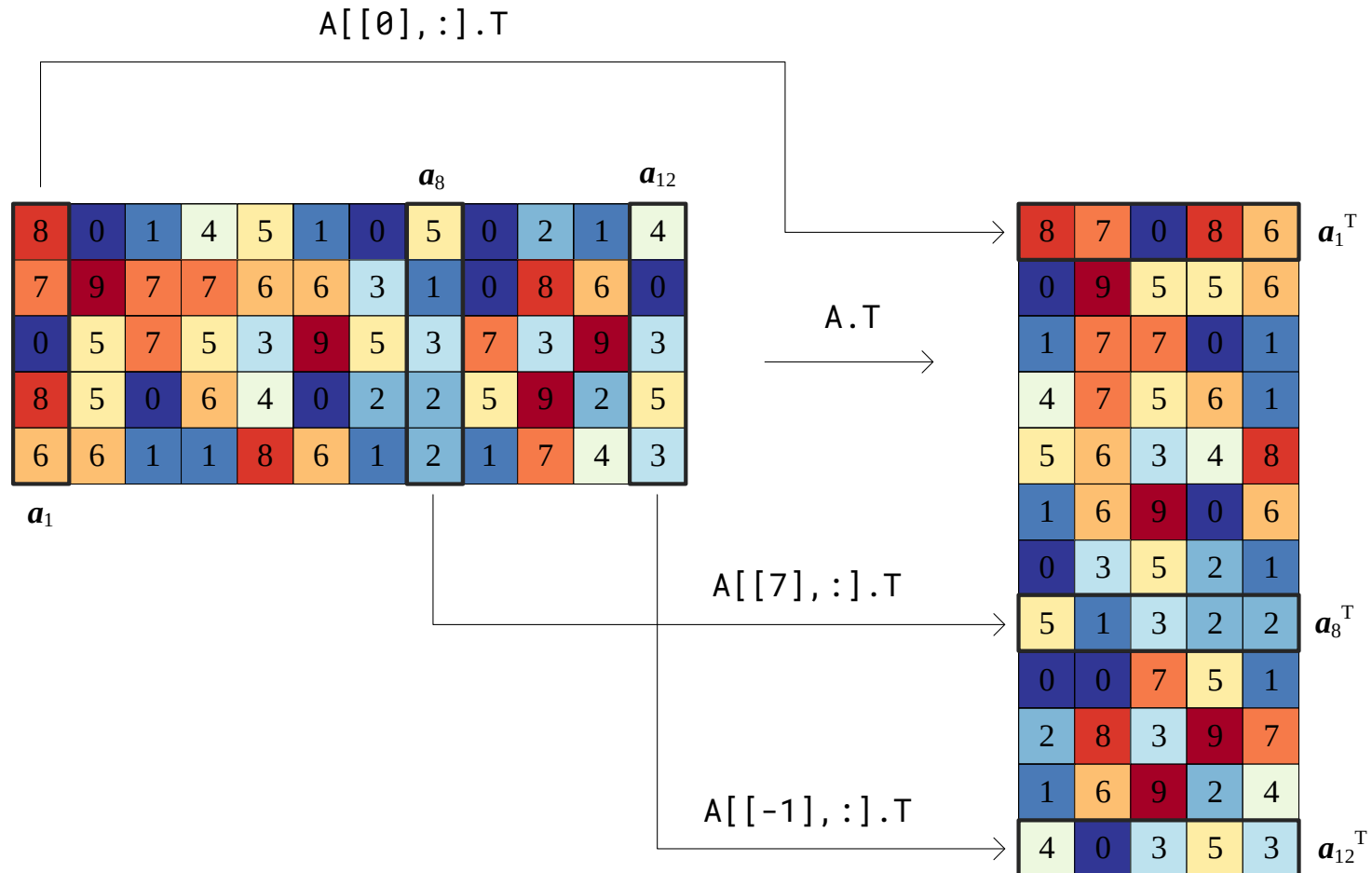
## Example: Transposing a Matrix by Column

$$A^T = [0,5; 3,4; 5,0]^T = [0,3,5; 5,4,0]$$

- The 1st entry of every column in A becomes the 1st entry of every row in  $A^T$
- The 2nd entry of every column in A becomes the 2nd entry of every row in  $A^T$

## Figure 3 · NumPy Transpose, Visualized: Columns → Rows

- $A[[0],:].T$ ,  $A[[7],:].T$ ,  $A[[-1],:].T$  each pull out column 1, 8, or 12 and transpose it into the matching row
- $A.T$  transposes the whole matrix, producing a new 12×5 array



## Transposing a 2D NumPy Array with .T

*Getting the transpose with .T, and comparing before/after with heatmaps*

- `np.random.seed(88)` fixes the random seed; `np.random.randint(0,10,(5,12))` generates a random 5×12 integer matrix A
- `A_T = A.T` computes the transpose of A in one step — the most common way to transpose a NumPy array
- `sns.heatmap(A[[0],:],...)` visualizes row 1 of A; `sns.heatmap(A[[0],:].T,...)` shows that same row turned into a column
- `cmap`, `square`, `cbar`, `annot`, `vmin/vmax`, `linecolor`, and `linewidths` control the heatmap's coloring, cell shape, and annotations

## 初始化

```
import numpy as np
import seaborn as sns
```

a

```
np.random.seed(88)
A = np.random.randint(0, 10, (5, 12))
```

## 计算矩阵 A 的转置

b

```
A_T = A.T
```

## 行向量的转置

c

```
sns.heatmap(A[[0],:], cmap = 'RdYlBu_r', square = True,
            cbar = False, annot = True, vmin = 0, vmax = 9,
            linecolor = 'w', linewidths = 0.25)
```

d

```
sns.heatmap(A[[0],:].T, cmap = 'RdYlBu_r', square = True,
            cbar = False, annot = True, vmin = 0, vmax = 9,
            linecolor = 'w', linewidths = 0.25)
```



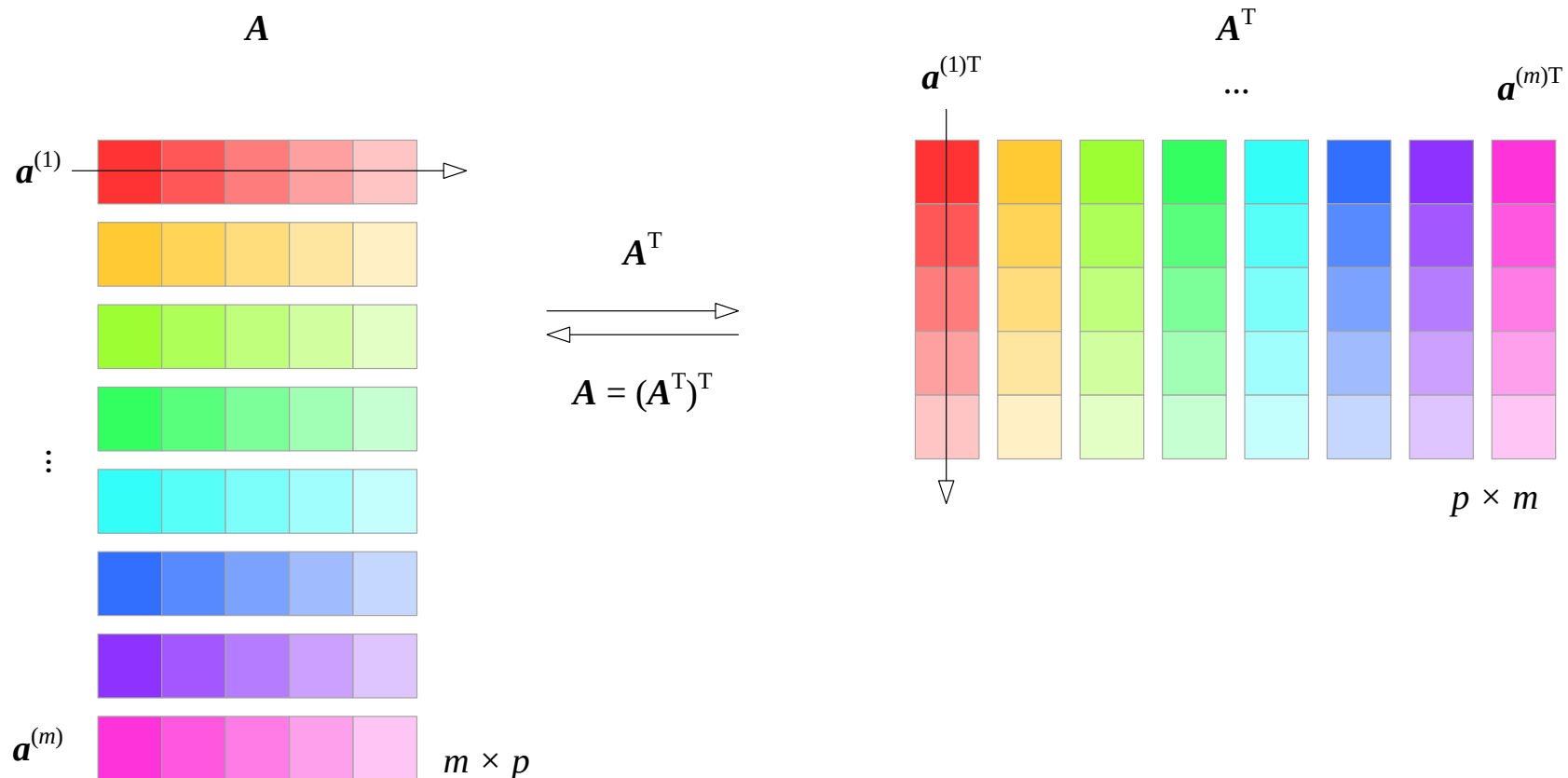
## Writing a Matrix as a Set of Row Vectors

$$\mathbf{A} = [\mathbf{a}^{(1)}; \mathbf{a}^{(2)}; \dots; \mathbf{a}^{(m)}], \text{ so } \mathbf{A}^T = [\mathbf{a}^{(1)T} \mathbf{a}^{(2)T} \dots \mathbf{a}^{(m)T}]$$

- Once  $\mathbf{A}$  is written as a set of row vectors, transpose turns each row vector into the matching column vector
- Row  $i$  of  $\mathbf{A}$  becomes column  $i$  of  $\mathbf{A}^T$ ; the relative order of entries — the shading gradient — never changes

## Figure 4 · Transpose Turns Rows into Columns

- Every original row vector becomes the matching column vector in  $A^T$
- Every entry in row  $i$  of  $A$  becomes the matching entry in column  $i$  of  $A^T$



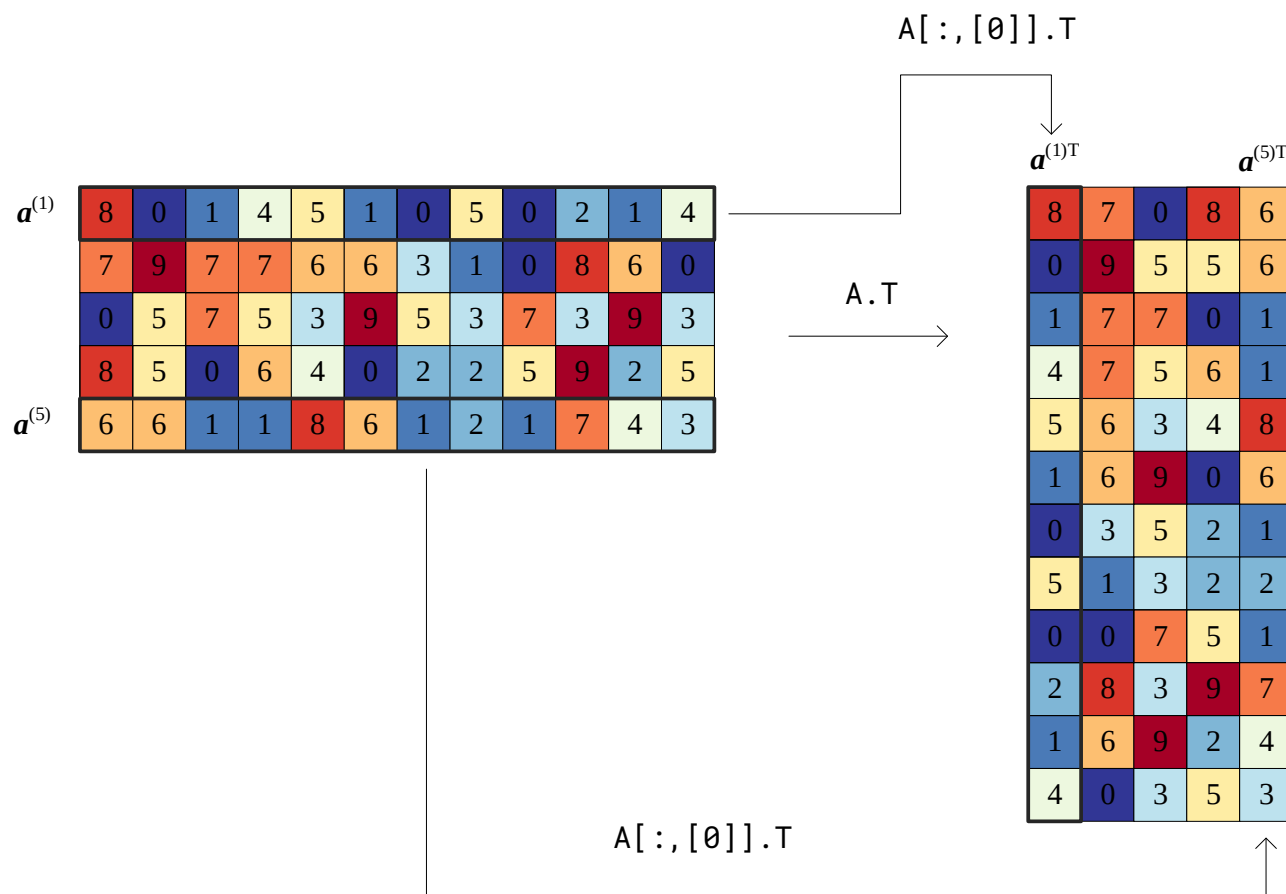
## Example: Transposing a Matrix by Row

$$A^T = [0,5; 3,4; 5,0]^T = [0,3,5; 5,4,0]$$

- The 1st entry of every row in A becomes the 1st entry of every column in  $A^T$
- The 2nd entry of every row in A becomes the 2nd entry of every column in  $A^T$

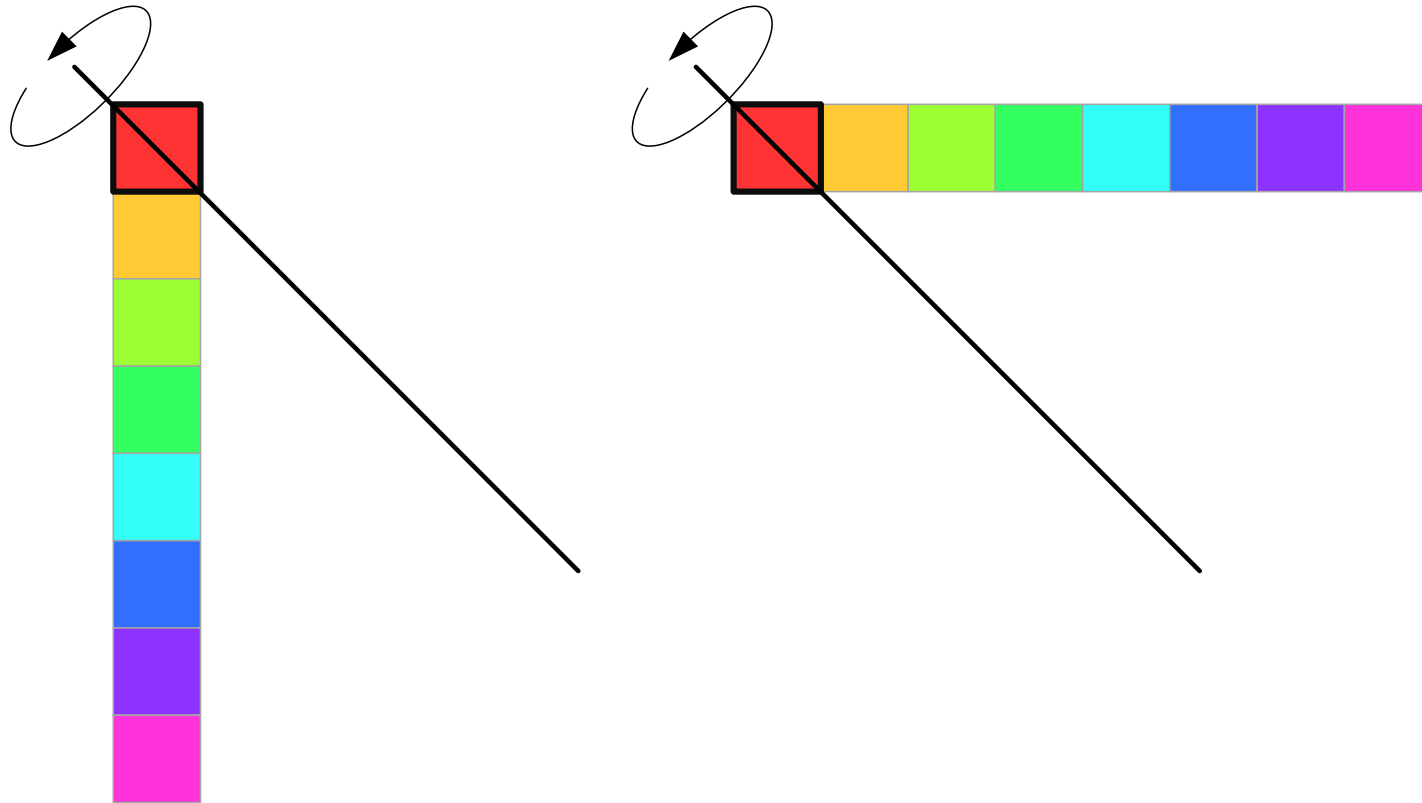
## Figure 5 · NumPy Transpose, Visualized: Rows → Columns

- $A[:, [0]].T$  pulls out column 1 and transposes it into a row vector, showing the other direction of the same idea
- LA\_02\_02\_01.ipynb also visualizes matrix A's row vectors transposing into column vectors



## Figure 6 · Row and Column Vectors Transpose into Each Other

- A row vector transposes into a column vector, e.g.  $[1,2,3]^T = [1;2;3]$
- A column vector transposes into a row vector, e.g.  $[1;2;3]^T = [1,2,3]$



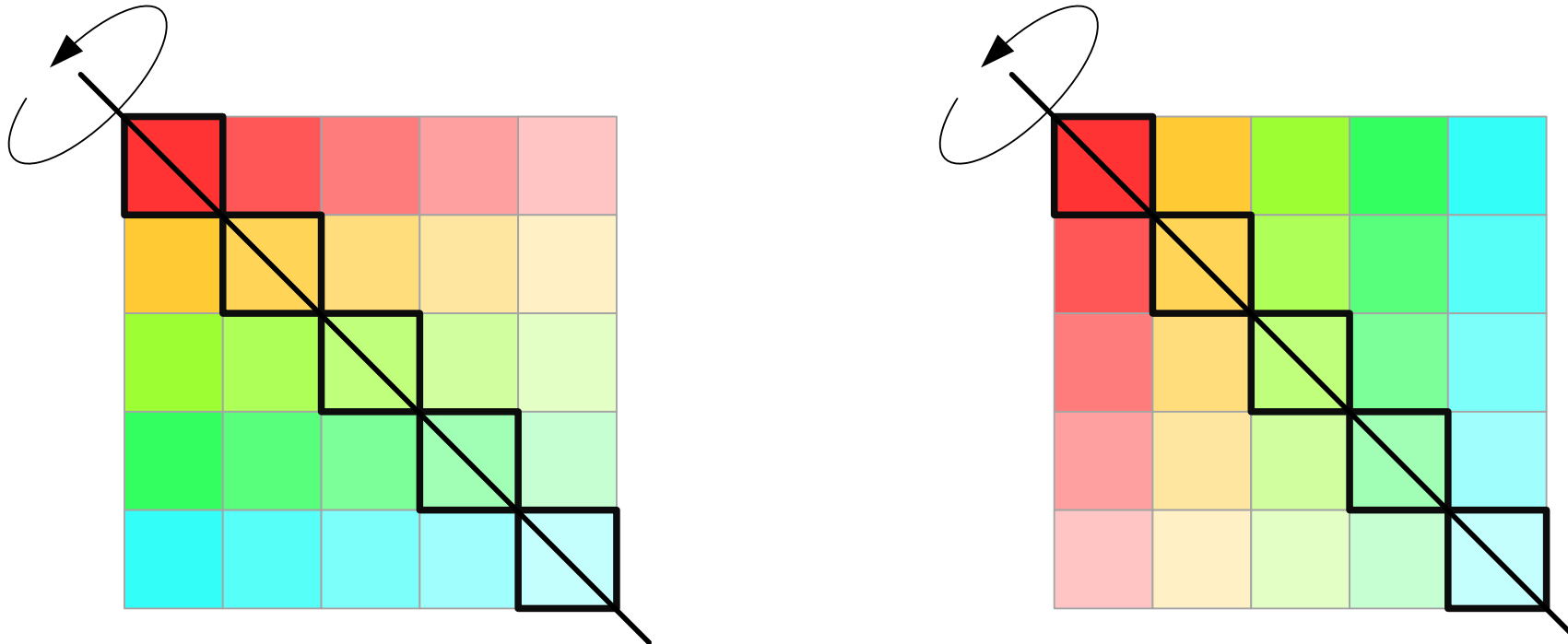
## Tall and Wide Matrices Swap Under Transpose

*A tall matrix (more rows than columns) transposes into a wide one, and vice versa*

- As in Figure 1, an  $m \times p$  matrix transposes into a  $p \times m$  matrix — rows and columns swap counts, and the shape changes accordingly
- The core rule never changes: transpose always turns rows into columns and columns into rows, with entries keeping their relative positions

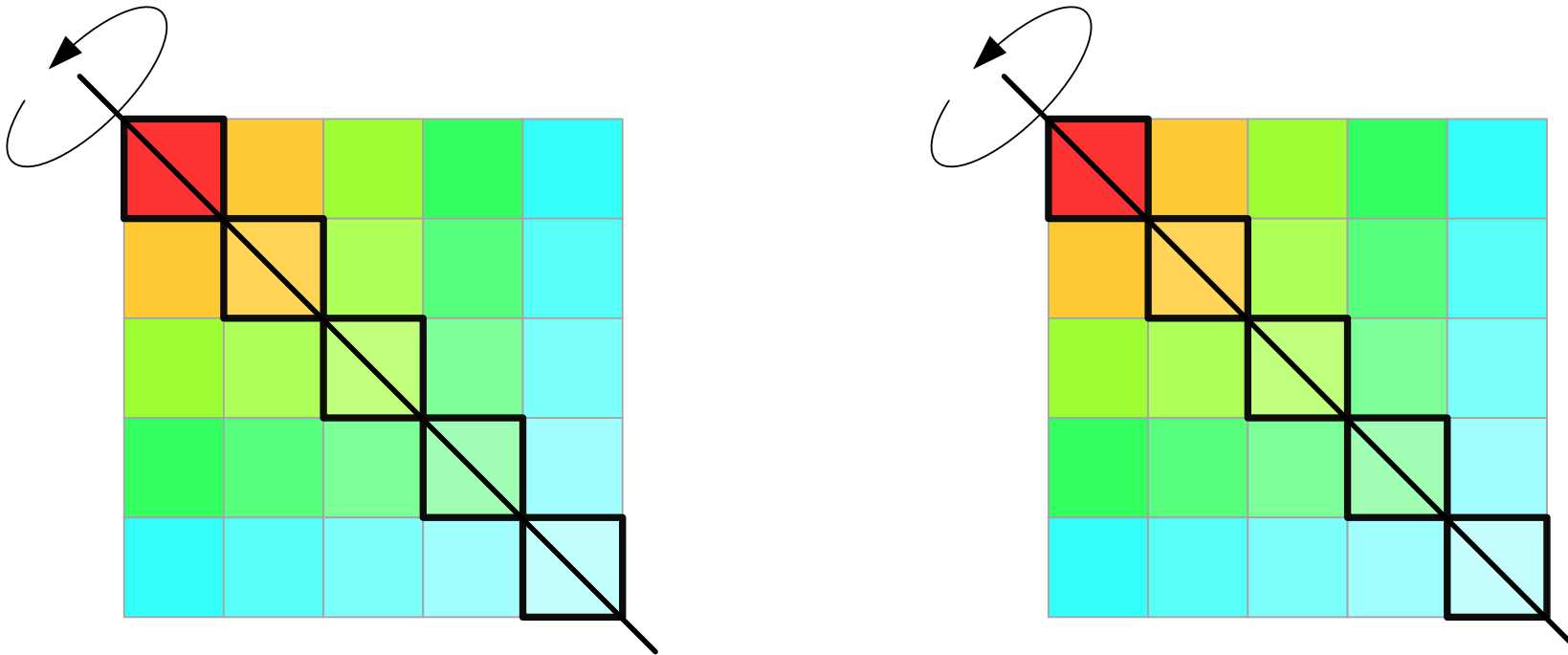
## Figure 7 · The Transpose of a Square Matrix Is Still Square

- A square matrix (equal rows and columns) keeps its shape after transpose — still square
- But entries mirror around the main diagonal, e.g.  $[1,3;2,4]^T = [1,2;3,4]$



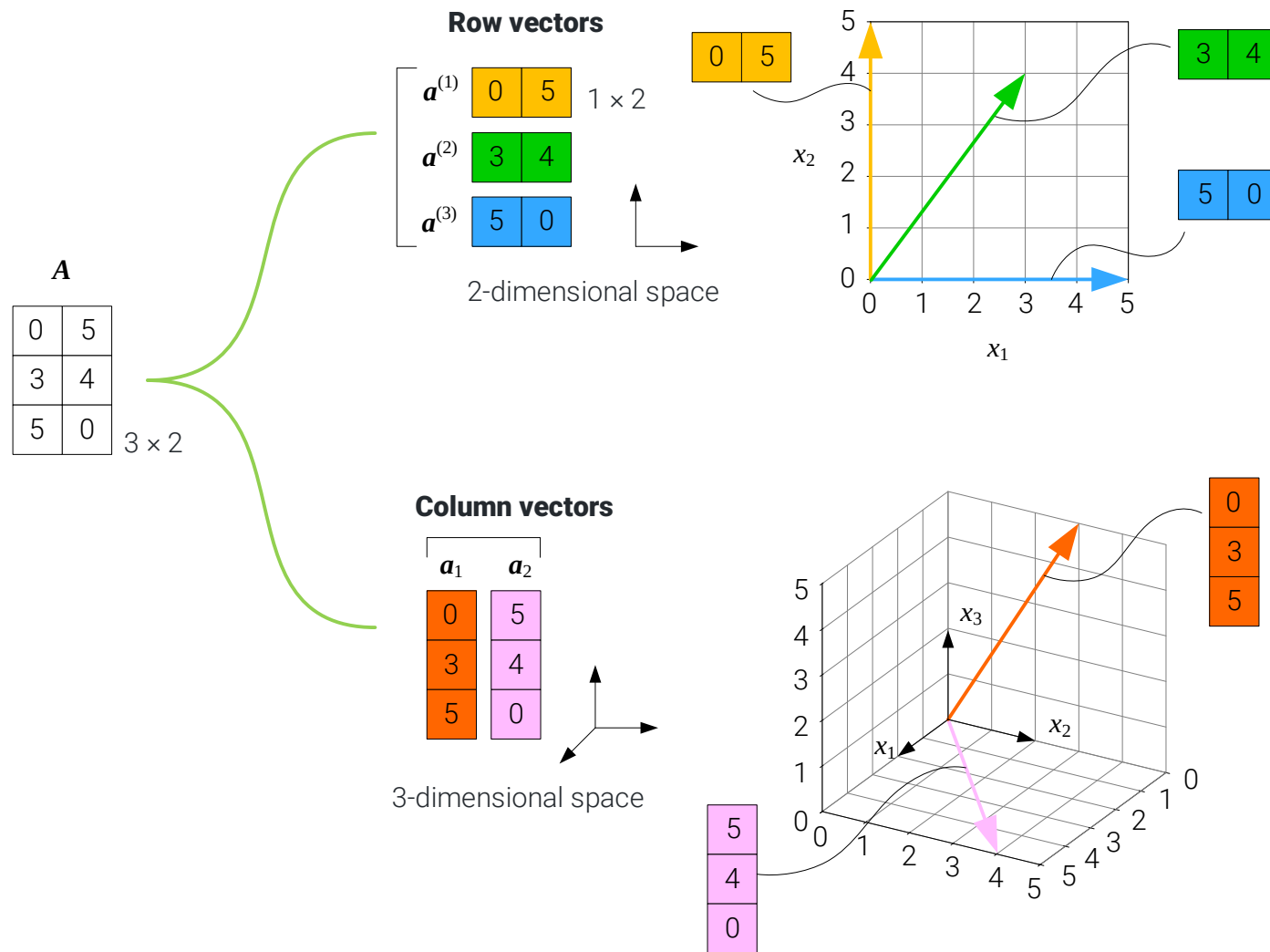
## Figure 8 · A Symmetric Matrix Equals Its Own Transpose

- A symmetric matrix satisfies  $\mathbf{A} = \mathbf{A}^T$  — its transpose is identical to itself
- A matrix's shape is the topic of the next section, 2.3 Shape of Matrix



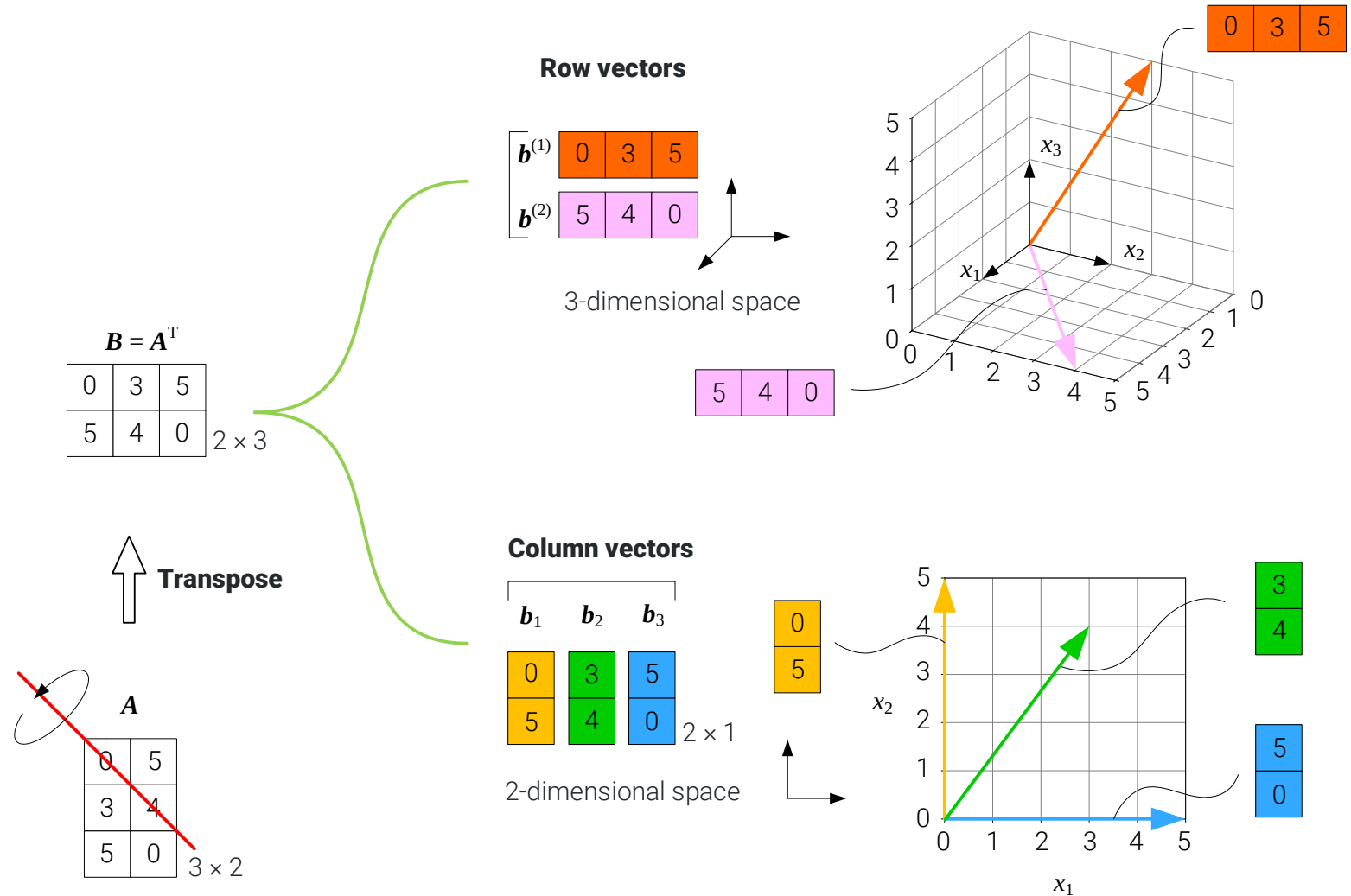
## Figure 9 · Row and Column Vectors of Matrix A (Geometric View)

- A's row vectors can be seen as arrows in a plane; A's column vectors as arrows in 3D space
- Image from Beauty of Visualization, showing how matrices correspond to geometric vectors



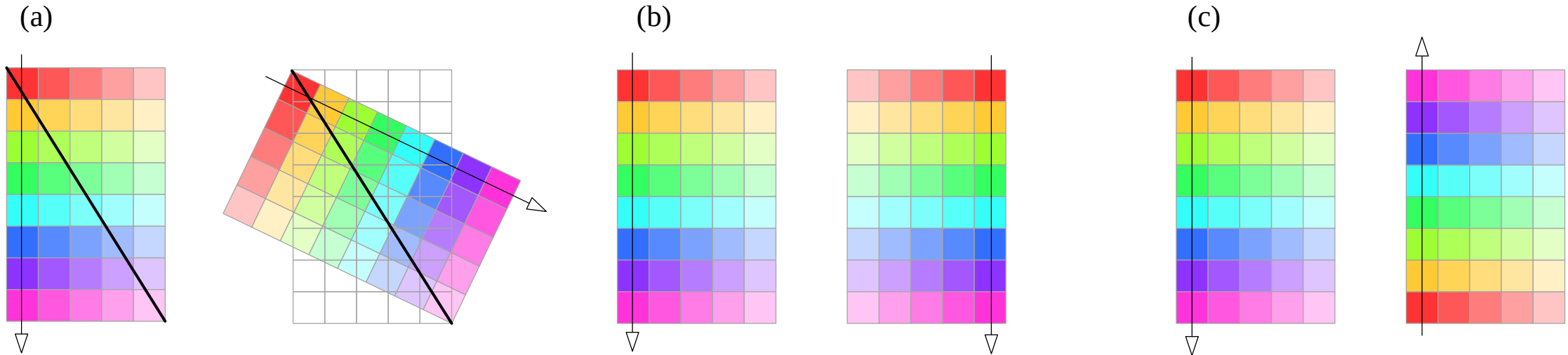
# Figure 10 · Row and Column Vectors of the Transpose (Geometric View)

- After transpose, the former column vectors become row vectors:  $A^T$ 's row vectors are now arrows in 3D space
- $A^T$ 's column vectors become arrows in a plane, matching  $A$ 's row vectors one for one



## Figure 11 · Three Misconceptions About Transpose

- (a)(b)(c) Transpose is not a diagonal mirror, not a left-right mirror, and not an up-down mirror
- Transpose has one exact meaning: entries in row  $i$  move strictly to column  $i$



## Property One: Double Transpose Restores the Original Matrix

$$(\mathbf{A}^T)^T = \mathbf{A}$$

- Transpose swaps rows and columns; applying it a second time swaps them right back
- For example,  $\mathbf{A}$  transposes into  $\mathbf{A}^T$ , and transposing  $\mathbf{A}^T$  again gives back  $\mathbf{A}$

## Property Two: Transpose Commutes with Addition

$$(A + B)^T = A^T + B^T$$

- Matrix addition works entry by entry, and transpose only moves entries around — so add-then-transpose and transpose-then-add give the same result
- This requires **A** and **B** to have the same shape; the same holds for matrix subtraction

## Property Three: Scalar Multiplication Is Unaffected by Transpose

$$(kA)^T = kA^T$$

- A scalar  $k$  scales every entry the same way; transpose only relocates entries without changing their values, so the two operations commute
- Transpose interacts with matrix multiplication and matrix inverses in further ways, covered in later sections

## Key Terms in This Section

**Transpose** *Swapping a matrix's rows and columns to get a new matrix, written  $A^T$*

**Unchanged Main Diagonal** *The entries  $a_{i,i}$  that stay in place before and after transpose*

**Transpose of a Column Vector** *A column vector transposes into a row vector*

**Transpose of a Square Matrix** *Shape unchanged; entries mirror around the main diagonal*

**Double Transpose**  $(A^T)^T = A$

**The  $A^T$  Notation** *This book always writes transpose as  $A^T$ , matching NumPy's `.T`*

**Transpose of a Row Vector** *A row vector transposes into a column vector*

**Tall / Wide Matrix** *Transpose converts one into the other*

**Symmetric Matrix** *A square matrix satisfying  $A = A^T$*

**Distributivity of Transpose**  $(A+B)^T = A^T+B^T$ ;  $(kA)^T = kA^T$

## GO FURTHER

# Companion Code and Further Reading

### Code 1 · LA\_02\_02\_01.ipynb

NumPy transpose operations; visualizing columns→rows and rows→columns

### Further Reading · LA\_02\_02\_02.ipynb

Transposing matrices of different shapes — self-study

### Recommended Reading · Beauty of Visualization

Source of this section's Figures 9 and 10 (geometric view)

### Open-source Repository

Code and image assets for the whole book: <https://github.com/Visualize-ML>

## Questions Q1 – Q9

**Q1** LA\_02\_01\_01.ipynb repeatedly calls `seaborn.heatmap()` to draw similar heatmaps; refactor the repeated code into a single Python function for later reuse.

---

**Q3** Write Python code to verify that "double transpose restores the original matrix"; use `numpy.array_equal()` to check whether two matrices match.

---

**Q5** Write Python code to verify that "scalar multiplication is unaffected by transpose."

---

**Q7** Generate a random  $6 \times 3$  integer matrix  $A$ , and write code verifying that  $A.T.shape$  equals  $(A.shape[1], A.shape[0])$ .

---

**Q9** Generate a random  $5 \times 5$  matrix  $A$ , build  $B = A + A.T$ , and verify whether  $B$  is always symmetric — explain why.

---

**Q2** Generate a random  $5 \times 8$  matrix, extract each of its rows, and then use `numpy.block()` to reassemble those row vectors back into the original matrix.

---

**Q4** Write Python code to verify that "transpose commutes with matrix addition."

---

**Q6** Write Python code that attempts to recreate Figures 9 and 10.

---

**Q8** Write a function `check_symmetric(A)` that determines whether a square matrix is symmetric (hint: compare  $A$  with  $A.T$ ).

---

## Questions Q10 – Q18

**Q10** Generate a random 4×4 matrix A, and write code verifying that its main-diagonal entries are unchanged after transpose.

**Q12** Generate a random 3×4 matrix A and a 4×3 matrix B, verify whether  $A + B.T$  is commutative, and explain why the shapes must match for the addition to work.

**Q14** Use `seaborn.heatmap()` to draw matrix A and its transpose  $A.T$  side by side, and observe how the color layout mirrors between them.

**Q16** Generate a random 8×8 matrix A, and write code verifying whether `numpy.diag(A)` always equals `numpy.diag(A.T)` — explain why.

**Q18** Generate two random matrices A and B of the same shape, and verify whether  $(A - B).T$  equals  $A.T - B.T$ .

**Q11** Write a function `row_to_col(v)` that takes a 1D row vector and returns its transposed column vector (hint: `reshape` or `[:,None]`).

**Q13** Write code verifying that a tall matrix (more rows than columns) always transposes into a wide one (fewer rows than columns); test at least 3 different shapes with random matrices.

**Q15** Write functions `is_row_vector(x)` and `is_col_vector(x)` that determine whether a NumPy array is a row vector or a column vector, respectively.

**Q17** Write a function `transpose_by_loop(A)` that implements matrix transpose by hand with a nested for loop, without calling  $A.T$  or `numpy.transpose()`, and compare it against  $A.T$ .

## Questions Q19 – Q26

**Q19** Build a 5×5 identity matrix with `numpy.eye(5)`, verify that its transpose equals itself, and state which special class of matrix this belongs to.

**Q21** Generate a random 3×3 matrix A, and verify the relationship between matrix multiplication and transpose: is `A @ A.T` always symmetric?

**Q23** Load the Iris dataset with Pandas, convert it to a NumPy matrix, transpose it, and explain which direction of the matrix the "sample" and "feature" dimensions correspond to before and after.

**Q25** Write code verifying whether the transpose operation changes the memory a NumPy array occupies (hint: use `numpy.shares_memory()` to check whether A and A.T share memory).

**Q20** Write code comparing `numpy.transpose(A)` with `A.T` — are their results and usage completely identical?

**Q22** Write a function `block_diag_check(blocks)` that takes several square matrices, builds a block-diagonal matrix from them, and verifies that its transpose equals the block-diagonal matrix reassembled from each block's transpose.

**Q24** Generate a random 2×3×4 three-dimensional array, call `numpy.transpose(arr, axes=(1,0,2))`, and observe and explain how transposing a multidimensional array differs from transposing a 2D matrix.

**Q26** Bringing it together, write a small program that, given any matrix, prints its transpose, checks whether it is square, checks whether it is symmetric, and verifies that double transpose restores the original matrix.

## 10 Prompts to Explore Further (1/2)

- 1 Explain why this book writes transpose as  $A^T$  rather than  $A'$ , and how that relates to NumPy's `.T` notation.
- 2 Use an everyday example to explain why "transpose is not a mirror" — like swapping "day" and "period" in a class schedule.
- 3 Prove that a symmetric matrix must be square; conversely, must a square matrix be symmetric? Give a counterexample.
- 4 Why does  $(A+B)^T = A^T+B^T$  hold, but  $(AB)^T$  generally does not equal  $A^TB^T$ ? (Hint: think ahead to the section on matrix multiplication.)
- 5 Explain why diagonal entries stay in place under transpose, and how that helps in checking whether a matrix is symmetric.

## 10 Prompts to Explore Further (2/2)

6

Design a small Python experiment verifying how tall and wide matrices swap shapes under transpose.

---

7

In the geometric view of Figures 9 and 10, explain what dimensional space a matrix's row and column vectors live in before and after transpose, and why the dimensions swap.

---

8

Give a data-science scenario explaining why we often need to transpose a data matrix — such as switching between samples×features and features×samples.

---

9

Explain the reasoning behind misconception (b), "transpose is not a left-right mirror": why does column 1 always become row 1, rather than moving to the far right?

---

10

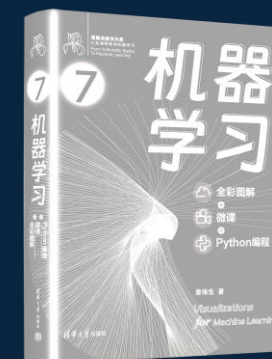
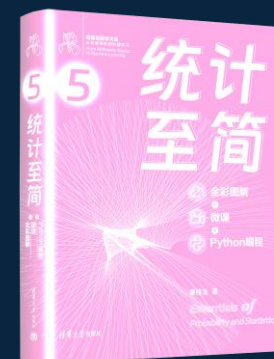
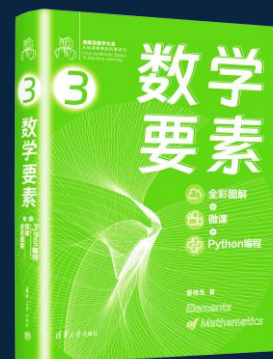
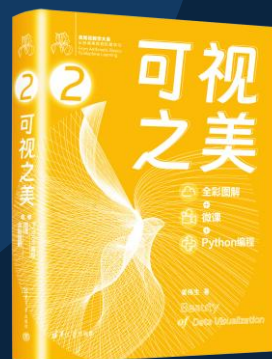
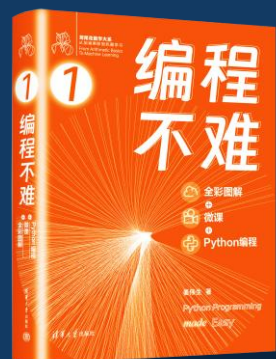
If a matrix satisfies  $A = -A^T$  (a skew-symmetric matrix), what would you expect of its main-diagonal entries? Verify your guess with Python.

# 数学

数学基础

线性代数

概率统计



Python编程

数据可视化

工具

数据分析  
图和网络

回归、降维  
分类、聚类

实践

